

A Survey of Open-Source UAV Autopilots

Nourdine Aliane 

Department of Engineering, Universidad Europea de Madrid, 28670 Villaviciosa de Odón, Madrid, Spain; nourdine.aliane@universidadeuropea.es

Abstract: This survey provides a comprehensive comparison of prominent open-source unmanned aerial vehicle (UAV) autopilots, focusing on their hardware compatibility, software features, and communication protocols. Additionally, it assesses the impact of these autopilots on research and education by examining their potential for integration with companion computers, compatibility with robot operating system (ROS) middleware and the MATLAB/Simulink environment, and the availability of simulation-in-the-loop (SITL) and hardware-in-the-loop (HITL) simulation tools. The paper concludes with a discussion of the advantages and disadvantages of these leading open-source autopilots.

Keywords: unmanned aerial vehicle (UAV); open-source autopilots; ROS; MATLAB/Simulink

1. Introduction

In the last decade, there has been a significant increase in the use of unmanned aerial vehicles (UAVs). This surge is attributed to technological advancements, such as the miniaturization of components, the availability of GPS systems and high-resolution cameras, among others, which have made drones more affordable for civilian use.

An autopilot is an embedded computer that runs software that provides the necessary control and guidance of UAVs. Key tasks carried out by an autopilot include platform stabilization to maintain the UAV's attitude and heading, mission planning along way points, deploying failsafe actions when necessary, connecting with ground control stations for telemetry data management, and so on.

Over the years, a variety of autopilots have been developed for civil, military, research, and educational applications. Some autopilots are designed as plug-and-play units, focusing on users who do not require in-depth knowledge of internal programming. Open-source autopilots, in contrast, enable developers to customize the autopilot internal workings, and their hardware and software design are publicly accessible and can be modified. The significance of open-source autopilots lies in their flexibility, cost-effectiveness, and the support of collaborative community of developers, fostering innovation and customization.

This paper is a survey of prominent open-source autopilots. It explores their impact on research and education and gathers evidence of their capabilities in streamlining UAVs' application development, including their potential for pairing with companion computers, their compatibility with ROS middleware and/or the MATLAB/Simulink environment, and the availability of simulation-in-the-loop (SITL) and hardware-in-the-loop (HITL) tools.

The rest of the paper is structured as follows. Section 2 outlines the methodology followed in this survey. Section 3 provides the key features of the prominent open-source autopilots. Section 4 provides an analysis of the tools used for UAVs' application deployment. Section 5 presents a discussion about the pros and cons of each of the prominent autopilots. Finally, Section 6 draws some conclusions.

2. Methodology

This survey synthesizes the features and technical information of prominent open-source autopilots from the websites of the projects developing these technologies. Further-



Citation: Aliane, N. A Survey of Open-Source UAV Autopilots. *Electronics* **2024**, *13*, 4785. <https://doi.org/10.3390/electronics13234785>

Academic Editors: Sergio Trilles Oliver, Mahmut Reyhanoglu, Marco Del-Coco, Pierluigi Carcagni, Ditsuhi Iskandaryan and Xin Wang

Received: 2 October 2024
Revised: 23 November 2024
Accepted: 2 December 2024
Published: 4 December 2024



Copyright: © 2024 by the author. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

more, the survey evaluates the impact of the prominent autopilot on research and education by gathering evidence on their potential for pairing with companion computers, on their compatibility with ROS and MATLAB/Simulink Pixhawk Support Package (PSP), and the availability of SITL and HITL simulation tools. This evaluation is based on carrying out searches in databases such as IEEE Xplore, ScienceDirect, the ACM Digital Library and MDPI, which encompass a wide range of academic journals and conference proceedings relevant to our study. To focus on recent developments in open-source autopilots, this survey considers only articles published within the last ten years. The aim is not to provide an exhaustive literature review on all open-source autopilots, but to establish the prevalence of certain autopilots in specific areas.

3. Prominent Open-Source Autopilots

Over the past two decades, numerous open-source autopilot projects have significantly advanced UAV technology. Several open-source autopilot projects like MatrixPilot, Baseflight, TauLabs, OpenPilot, Cleanflight, MultiWii, and dRonin are discontinued, when others such as Ardupilot/APM [1], Pixhawk/PX4 [2,3], Paparazzi [4], LibrePilot [5], Betaflight [6], or iNAV [7] remain relevant today, and have gained a wide acceptance among developers and hobbyists. These prominent open-source autopilots have been refined by active communities of developers worldwide, each bringing unique features and expanding compatibility to stay at the forefront of UAV technology. They support a wide variety of vehicle types, including multi-copters, fixed-wing aircraft, rovers, and aquatic drones, showcasing their adaptability to diverse applications.

Ardupilot/APM was originally developed by Jordi Muñoz and Chris Anderson in 2007, as part of the DIY Drones community and as an offshoot of the Arduino project, and was the first widely used hardware platform. It was one of the first open-source autopilots to gain widespread use. Ardupilot stands out for its compatibility with numerous vehicle types, such as multi-copters, fixed-wing aircraft, rovers, and aquatic vehicles. Key releases like ArduPlane, ArduCopter, and ArduRover have tailored the autopilot for specific vehicle types. Today, it is maintained by the ArduPilot development team and continues to be widely used in both hobbyist and professional UAV applications.

Pixhawk/PX4 was initiated by Lorenz Meier in 2008 at ETH Zurich [8]. The hardware platform, Pixhawk, was released in collaboration with 3D Robotics. The PX4 autopilot software, developed as a highly modular and flexible flight control system, supports a wide array of vehicles, including multi-copters, fixed-wing aircraft, VTOL platforms, rovers, and boats. The architecture of PX4 allows for complex configurations and advanced features, making it suitable for both simple and highly complex UAV projects. PX4 is now managed by the Dronecode Foundation with support from a large community and industry partners.

Paparazzi UAV began in 2003 at ENAC (École Nationale de l'Aviation Civile) in France [9]. As one of the earliest open-source autopilot projects, it focuses on academic research and development. It supports a range of vehicle types, including multi-copters, fixed-wing aircraft, and even some experimental platforms like hybrid vertical-take-off and landing (VTOL) vehicles. Over the years, numerous versions and updates have enhanced its capabilities.

LibrePilot, created in 2015, is a fork of the discontinued OpenPilot project, focusing on reliability and user-friendly interfaces. Although primarily focused on multi-copters, it also supports fixed-wing aircraft and experimental platforms. Each release aims to improve flight performance, user interface, and configuration support.

Betaflight, derived from Cleanflight in 2015, is tailored for racing drones and acrobatic flying. Known for its superior flight performance and responsiveness, Betaflight has become the preferred firmware for FPV racing, with rapid development and frequent updates enhancing its capabilities. The streamlined architecture of Betaflight makes it exceptionally effective for its targeted application but less versatile for other vehicle types. iNAV, another fork of Cleanflight, provides advanced navigation capabilities for multi-copters, fixed-wing aircraft, and rovers. It focuses on reliability and precision control,

making it a preferred choice for users requiring advanced navigation features. Table 1 summarizes the key features of prominent open-source autopilots, and Table 2 summarizes the supported vehicles.

Table 1. List of active and discontinued open-source autopilots.

Autopilot	Year	Country	Discontinuity
Paparazzi	2003	France	Active
MatrixPilot	2008	USA	2016
APM (ArdupilotMega)	2009	USA	Active
MultiWii	2010	USA	2016
PX4 (Pixhawk)	2011	Switzerland	Active
Baseflight	2012	USA	2014
TauLabs	2012	USA	2016
OpenPilot	2013	USA	2015
Cleanflight	2014	USA	2020
LibrePilot	2015	USA	Active
Betaflight	2015	USA	Active
dRonin	2015	USA	2019
iNAV	2016	USA	Active

Table 2. Prominent autopilots and supported vehicles.

Autopilot	Copters	Fixed Wings	RotorCraft	Rovers	Aquatic
Paparazzi	Tri, Quad, Hexa, Octo	Yes	Yes	-	Yes
APM (Ardupilot)	Tri, Quad, Hexa, Octo	Yes	Yes	Yes	Yes
PX4 (Pixhawk)	Tri, Quad, Hexa, Octo	Yes	Basic	Yes	Yes
LibrePilot	Tri, Quad, Hexa, Octo	Yes	-	Basic	Basic
Betaflight	Tri, Quad, Hexa, Octo	Yes	Basic	-	-
iNAV	Tri, Quad, Hexa, Octo	Yes	Basic	Yes	Basic

3.1. Technical Features of Prominent Open-Source Autopilots

Open-source autopilots integrate diverse technical concepts that are essential for developing versatile UAV systems suitable for research, education, and recreational use. These concepts include open hardware compatibility, software features, communication protocols, and ground control stations (GCS). The following sections provide the main features, highlighting the significance of each component that can be found in prominent open-source flight controllers.

3.1.1. Open Hardware and Compatibility

Open hardware boards for UAVs provide the backbone for many open-source autopilot systems, offering flexibility, customization, and a range of features tailored to different UAV applications. Prominent open boards include the Pixhawk series, CUAV series, Lisa series, Apogee, Naze, CC3D, Kakute, and Omnibus, which are designed to cater to different needs, from hobbyist projects to professional research and commercial applications. Their open-source nature allows for extensive customization and integration with various sensors and software, making them highly versatile.

The Pixhawk series includes several versions (Pixhawk 3–6 or Pixracer, Cube), and is compatible with both Ardupilot, PX4 and paparazzi firmware. These boards are equipped

with powerful processors, multiple sensor interfaces, and support for advanced functionalities such as GPS navigation and real-time data logging. The CUAV series, including models like CUAV V5+ and CUAV Nora, builds on the Pixhawk legacy by adding enhanced features such as better IMUs, additional connectivity options, and improved power management. These boards are designed for high reliability and are often used in professional and industrial UAV applications. The boards are based on ARM STM32 [F1, F2, F3, F4] microcontrollers, depending on the version. The high-end version, such as Pixhawk 6X, is with a more powerful STM32H753 processor. The Lisa series, developed under the Paparazzi project, includes boards like Lisa/M and Lisa/S, which emphasize modularity and flexibility. These boards are particularly popular in academic and research settings due to their ease of integration and support for custom flight dynamics. Apogee boards, another product of the Paparazzi ecosystem, focus on high-performance applications and offer advanced features like dual IMUs for redundancy and improved flight stability. These boards are suitable for sophisticated UAV projects that require precise control and robust performance. Other notable open hardware designs include the CC3D board, developed initially for LibrePilot, and it provides good performance for basic drone control tasks; Naze32, another board developed to Betaflight ecosystem, and used in freestyle and racing drones for its robust flight performance; Kakute, designed with an onboard OSD and vibration-damping IMU, used for freestyle and racing drones, which are largely used with Betaflight and iNAV flight controllers; and the Omnibus board, designed with an integrated OSD and provides multiple versions (F3, F4, F7) for different needs. This multitude of hardware platforms allows developers to leverage the strengths of each autopilot system, providing flexibility and a wide range of options for various UAV applications. Table 3 summarizes the features of hardware platforms across major open-source autopilots, and Figure 1 shows the appearance of some autopilot models.



Figure 1. Appearance of some autopilot models. (a) CUAV V6X; (b) CUAV V5X; (c) Pixhawk cube; (d) Pixhawk 4; (e) Paparazzi Lisa/S; (f) Naze32.

Table 3. Hardware compatibility across major open-source UAV autopilots.

Board	Models	Autopilot	Main Features
Pixhawk	Pixhawk 3–6, Pixracer, Mini	PX4, Ardupilot, Paparazzi	Hobby, Research, Education
CUAV	CUAV V5+, CUAV X7	PX4, Ardupilot	High-End UAV, Industrial
Lisa	Lisa/M, Lisa/S	Paparazzi	Research, Education
Apogee	Apogee V.1.1	Paparazzi	Education
Naze	Naze32	Betaflight	Racing
CC3D	CC3D	LibrePilot	Hobby, DIY projects
Kakute	F4, F7	Betaflight, iNAV	Racing, Freestyle
Omnibus	F3, F4, F7	Betaflight, iNAV	Racing, Long-range drones

3.1.2. Software Considerations

The software considerations in open-source autopilots reflect their diverse technical features and capabilities, catering to a wide range of UAV applications. This section highlights essential aspects of autopilot firmware, including whether they utilize a specific real-time operating system (RTOS), the programming languages employed in their development, and their scripting capabilities.

Ardupilot was initially designed as a monolithic software, and then transitioned to use ChibiOS [10], an RTOS known for its robustness and suitability for embedded systems. It is developed in C/C++, and it has scripting capabilities using Dronkite API [11] and Lua [12]. PX4 is designed upon NuttX [13], a POSIX-compliant RTOS, which provides a scalable environment for real-time applications, facilitating multi-threading and advanced scheduling. Its middleware is written in C, and it allows for advanced scripting capabilities through MAVSDK API, using languages like C++, Python, or Swift [14]. Paparazzi in its last releases is also designed upon ChibiOs [15]. It is primarily written in C, and it allows for advanced scripting capabilities through Pprzlink toolkit [16], which supports scripting languages with C, OCaml and Python. LibrePilot uses FreeRTOS [17], a widely adopted RTOS in embedded systems, providing reliable task management. It is developed in C language, with a focus on embedded systems programming. Betaflight and iNAV do not use traditional RTOS, and they are designed upon a customized scheduler optimized for rapid execution of flight control loops. They are written in C, and they provide scripting capabilities with Python, through the YAMSPy protocol [18].

The PX4 software is released under BSD license, allowing software modification and redistribution under other licenses, and all the remaining autopilot software are released under the GNU-GPL license, allowing software modification but only permitting redistribution as long as the modified source code remains free.

3.1.3. Communication Protocols

Open-source UAV autopilots utilize a variety of communication protocols to ensure efficient data exchange between the UAV components and ground control systems (GCS). These protocols facilitate telemetry and control, each suited to the specific needs of the autopilot systems.

Ardupilot primarily uses MAVLink [19,20], a lightweight messaging protocol that supports telemetry and control functions. Enhanced security protocols known as MAVsec [21] are also used for missions requiring more secure communication. Additionally, Ardupilot utilizes the controller area network (CAN) protocol for high-speed communication between onboard components such as GPS units, sensors, and controllers. PX4 also relies on MAVLink for its primary communication protocol. It also employs UAVCAN, a protocol designed for robust and reliable communication in robotic applications, used to facilitate real-time data exchange between various UAV components. Additionally, PX4

uses real-time publish–subscribe (RTPS) for communication between PX4 and ROS, enhancing interoperability with robotic systems. Paparazzi utilizes the PprzLink toolkit [22] for telemetry data exchange between the UAV and GCS, and it employs the Ivy Bus, an event-driven communication protocol that enables components to subscribe to and publish messages on a shared communication bus within the system. LibrePilot employs UAVTalk [23], a custom binary protocol optimized for efficient communication between the UAV and GCS, and it has experimental support for MAVLink. Betaflight and iNAV use Multiwii serial protocol (MSP) [24], a lightweight binary protocol for configuration and telemetry communication between the flight controller and GCS, and both of them also have experimental support for MAVLink, allowing for telemetry and control through this protocol. Table 4 summarizes the software features of the prominent open-source autopilots.

Table 4. Main software features of prominent open-source autopilots.

Autopilot	Scripting Language	RTOS	Communication Protocol
Paparazzi	C, OCaml, Python	ChibiOs	Pprzlink
APM (Ardupilot)	Python, JavaScript, Lua	ChibiOS, NuttX	Mavlink
PX4 (Pixhawk)	C++, Python, Swift	NuttX	Mavlink
LibrePilot	-	FreeRTOS	UAVTalk
Betaflight	Python	-	MSP, Mavlink
iNAV	Python	-	MSP, Mavlink

3.1.4. Ground Control Stations Systems

GCS is a software that allows the operator to plan and execute autonomous missions, receive telemetry data in real-time, issue commands to control the UAV flight, and so on. In this respect, Ardupilot supports several GCS, including APM Planner2 [25], Mission Planner [26], Andropilot [27], or MAVProxy [28], where all of them share similar functionalities. The PX4 ecosystem primarily uses QGroundControl [29], known for its cross-platform compatibility and rich feature set, and it is also compatible with all the popular GCS software used with Ardupilot. Paparazzi, LibrePilot, Betaflight and iNAV have their GCS known as Paparazzi Center [30], LibrePilot GCS, Betaflight-Configurator and iNAV-Configurator, respectively, which provide streamlined interfaces for setting up and tuning the flight controllers. Table 5 provides the principal GCS and their compatibility with the different platforms.

Table 5. Prominent autopilots and supported vehicles type.

GCS	Platform
MissionPlanner	Windows, Mac os
APM Planner	Windows, Mac os, Linux
Mavproxy	Linux
Andropilot	Android,
QGroundControl	Windows, Mac os, Linux, Android,
Betaflight-Configurator	Windows, Mac os, Linux
iNAV-Configurator	Windows, Mac os, Linux

4. Analysis of Tools for UAVs' Applications Deployment

Open-source autopilot capabilities can be enhanced through various tools that streamline UAV application development. Among them, relevant tools include the use of companion computers for advanced processing, ROS framework for advanced instrumentation

integration, MATLAB/Simulink to advanced modeling and simulation tasks, and software-in-the-loop (SITL) and hardware-in-the-loop (HITL) for testing and validation. These tools, along with other provided APIs, collectively offer a rich ecosystem for developing advanced UAV applications.

4.1. Use of Companion Computers

A companion computer is an additional computing device that can be connected to an autopilot platform via serial or UDP connections. The choice of companion computers depends on the complexity of the tasks at hand, with several types of companion processors, such as Raspberry Pi, Nvidia Jetson, Odroid, Intel NUC, and BeagleBone, catering to various needs. Figure 2 shows an example of Pixhawk connected to a Raspberry Pi companion computer.

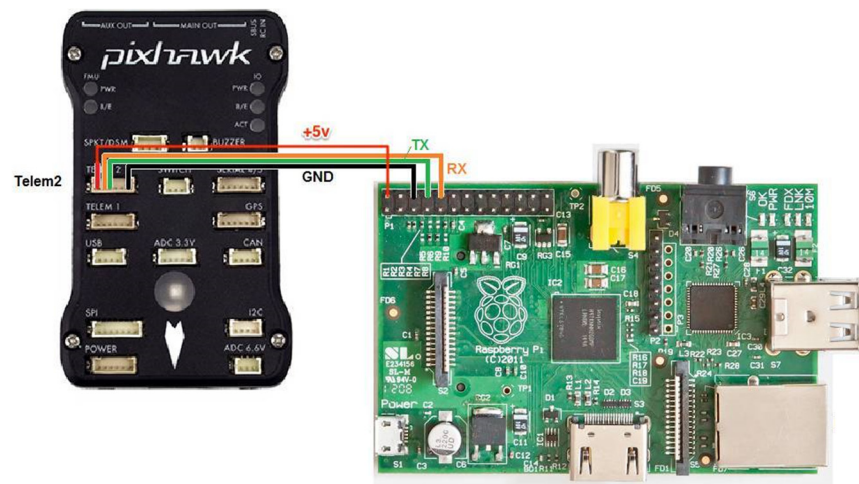


Figure 2. Pixhawk autopilot connected to a Raspberry Pi companion computer.

The Ardupilot and PX4 autopilots benefit greatly from companion computers running software like MAVROS [31], as communication is easily carried out using the MAVLink protocol. Additionally, the availability of the Dronekit API [11], a Python library, offers high-level features for interacting with Ardupilot and PX4 autopilots that enables developers to write Python code to monitor and control the drone's flight based on the MAVLink protocol. Another tool, MAVSDK [14], an API launched in 2018 that is considered the primary high-level interface for controlling MAVLink-based drones and provides bindings with multiple programming languages, including C++, Python, Swift, Rust, and Java.

There are numerous references showing the use of companion computers. For example, the paper [32] combines PX4 firmware with an Nvidia Jetson board for onboard processing of depth maps and collision avoidance algorithms. Similarly, the paper [33] proposes a modular autopilot solution for rapid UAV prototyping, combining PX4 with two companion computers (Raspberry Pi and NUC minicomputer) that implement functionalities directly in Simulink for high-level mission planning. Reference [34] reviews the integration of sensors, communication technologies, and machine learning in UAVs within PX4 firmware, highlighting the critical role of companion computers (Raspberry Pi, Jetson Tx1) in advanced data processing. Ardupilot is also combined with companion computers in many implementations, such as [35] that proposes a combination with a Raspberry Pi board to perform image processing of a monocular camera; or the paper [36] that describes the use of an Odroid board for controlling an autonomous surface vehicle; or the reference [37] that uses Nvidia Jetson TX for autonomous navigation using visual odometry for dynamic obstacle avoidance; and the work [38] that explores edge-accelerated UAV operations using a BeagleBone board. Mention also the use of Dronekit in [39] for autonomous drone operations, in the paper [40] which describes how to connect drones to Dronemap, a cloud-based management system for maps and images, or in [41] that presents

the development of an efficient swarm algorithm for terrain mapping using cooperative unmanned aerial vehicles. The use of MAVSDK is also notable, and the work [42] that presents a remote monitoring and navigation system for multiple drones, or the paper [43] that develops a secure UAV swarm communication system. Companion computers are also used with the Paparazzi autopilot. For example, Ref. [44] discusses the development of a Raspberry Pi-based portable, low-cost ground control station for drones, enhancing accessibility and usability for operators, or in [45] that describes an application of load-transportation using an Odroid board. Similarly, Ref. [46] proposes a design of an ad hoc network routing protocols involving multiple Paparazzi UAVs equipped with Raspberry Pi for communications, and Ref. [47] evaluates the security of open-source Linux operating systems for UAVs, highlighting potential vulnerabilities and proposing measures to enhance UAV cybersecurity. Regarding LibrePilot, as it focuses on simplicity and ease of use, the integration of companion computers is less common. However, there are some custom developments combining LibrePilot firmware and Raspberry Pi, such as in [48] describing a laboratory for hands-on drone experiments. Ref. [49] presents a framework for drone simulation testing, and the paper [50] deals with image processing for monitoring and surveillance. Ref. [51] presents a multilevel architecture interfacing a CC3D Revo flight controller running LibrePilot firmware with a Jetson Nano, which is also used in [52] for implementing precise maneuvering using Raspberry Pi aerial vehicles. In contrast, Betaflight and iNAV are used, respectively, in high-performance drone racing and autonomous flight, where the need for companion computers is minimal. However, a few papers address this aspect, such as the reference [53] combining Betaflight firmware with a Jetson Tx2 to carry out time-optimal planning for quadrotor waypoint, or the paper [54] presenting iNAV flight controller combined with a Raspberry Pi for implementing a security solution for UAVs.

4.2. ROS Integration

Developers often prefer using the Robot Operating System (ROS) middleware [55], an open-source framework that facilitates the development of robot software through its modular architecture. This architecture allows for easy integration of sensors, actuators, and various hardware components, while offering many pre-built algorithms and models for common robotics tasks.

The Ardupilot and PX4 ecosystems can be integrated with ROS via MAVROS [31], a tool that translates MAVLink messages into ROS messages. Typically, ROS and MAVROS are installed on a companion computer that communicates with the autopilot firmware. ROS has evolved into ROS2, where communication with Ardupilot and PX4 firmware is performed over the XRCE-DDS bridge [56], enabling efficient data exchange between the flight controller and companion computers. For instance, the paper [57] highlights the flexibility and robustness of Ardupilot when combined with ROS for sophisticated control and simulation tasks, and Ref. [58] showcases how ROS modules extend Ardupilot's capabilities to support advanced communication among multiple drones. Ref. [59] discusses the seamless integration of PX4 with ROS for drone control, and Ref. [60] illustrates a practical application of PX4 and ROS to achieve autonomous quadrotor landings. In the Ref. [61] is presented a distributed autonomous flight system that utilizes ROS and 3D position tracking to enhance UAV operations; in [62] issues related to securing UAV communications with a custom method implemented in ROS are described, and the work [63] introduces a testbed for UAV swarm simulation that leverages ROS and PX4 firmware for prototyping and basic research in drone swarming.

On the other hand, Paparazzi, LibrePilot, Betaflight, and iNAV do not offer the same level of support for ROS as Ardupilot and PX4, although several initiatives have been undertaken to bridge this gap. For instance, the "pprzros" project [64] aimed to interface Paparazzi with ROS, but the repository does not show recent activity. Similarly, Betaflight has a project called "SBus bridge" [65] for controlling Betaflight via ROS. In these projects, developers had to write custom nodes that convert autopilot telemetry data into ROS mes-

sages, which require understanding the communication protocols used by each autopilot (UAVTalk for LibrePilot, MSP for Betaflight and iNAV) and implementing translators to publish the data on ROS topics. Additionally, a search for remarkable papers does not yield significant results, except for a few singular papers, such as [66] which describes an experimental setup using LibrePilot and ROS to control an airship formation for animal motion capture, or [67] which describes an experimental setup using iNAV with ROS employing a monocular event-based camera for obstacle detection and avoidance.

4.3. MATLAB/Simulink and Pixhawk Support Package

The MATLAB Pixhawk Support Package (PSP) [67] toolbox enables developers to interact with PX4 autopilots by accessing Pixhawk peripherals from Simulink, reading data from connected sensors in real-time, and logging these data for post-flight analysis. Another method involves using the MAVLink communication protocol, where MATLAB scripts can be written to send and receive messages, thereby enabling real-time interaction with the PX4 firmware. Furthermore, developers can deploy control algorithms for UAVs, test and validate them without requiring physical hardware. For example, developers can perform UAV dynamic model simulations, design controllers and estimators such as PID controllers, Kalman filters, and path planning algorithms using MATLAB's built-in functions and tools. Developers can also generate C/C++ code tailored specifically for PX4 autopilot boards using MATLAB Coder, and the generated code can be deployed as part of the PX4 autopilots, and developers can replace the PX4 native control modules (position, attitude, and rate control) with user-defined control algorithms.

The scientific literature is abundant with papers illustrating the impact of combining MATLAB/Simulink with Pixhawk and PX4 platforms, showcasing their significant contributions to UAV research, education, and practical applications. For example, Ref. [68] presents RflySim, a multi-copter platform designed for education and research, leveraging Pixhawk hardware and MATLAB/Simulink PSP for UAV development, and the book [69] presents a series of step-by-step experiments and guidance for developing and controlling multi-copters, making it an invaluable resource for both researchers and educators in UAV technology. Ref. [70] describes the standard PID controller implementation for a quadcopter using MATLAB/Simulink, where Simulink models are converted to C/C++ using MATLAB Embedded Coder, and the resulting code is uploaded into the Pixhawk autopilot; paper [71] describes the deployment of a sliding mode control for a quadcopter, underlining the synergy between MATLAB/Simulink and PX4 for sophisticated control tasks. Ardupilot can also be connected to MATLAB, as detailed in the official documentation [72], and several papers illustrate this possibility. For example, the paper [73] demonstrates the use of MATLAB connected to a swarm of Ardupilot-based underwater vehicles through the MAVLink protocol; another work [74] describes the use of MATLAB to develop a flight data analyzer for Ardupilot-based autopilots, or the work [75] that proposes a software architecture operating under Linux, combining MATLAB and ROS to communicate with Ardupilot, aimed at testing UAV control algorithms. Regarding Paparazzi, LibrePilot, Betaflight, and iNAV, no remarkable papers have been found that confirm the possibility of connecting them with MATLAB/Simulink, as this integration is not straightforward and often requires significant programming effort to achieve reliable communication and data handling.

4.4. SITL and HITL Simulations

Aerial vehicles are more delicate than common ground vehicles, so control algorithms and software development are typically tested using simulation tools such as SITL and HITL. In the SITL context, the autopilot firmware is executed on a host computer, and it controls a simulated vehicle, which could be running on the same computer or on different ones. Developers interact with simulated vehicles using special protocols, such as MAVLink in the Ardupilot and PX4 ecosystems. In HITL simulations, the firmware runs on its native autopilot hardware, and it controls a real testbed or a simulated vehicle that provides sensor

feedback to the flight control stack. HITL simulations provide a realistic interaction with a real or virtual environment, enabling developers to test most of the flight code on real hardware with the benefit of a safe testing environment.

Several open-source SITL and HITL simulation tools are available, with the most important being Ardupilot-SITL [76], Gazebo [77], FlightGear [78], jMavSim [79], AirSim [80], JSBSim [81], and Webots [82]. These simulators are widely utilized in education and research to facilitate understanding of UAV technology. Here are the main simulators in the UAV field.

Ardupilot-SITL [76]: It runs the Ardupilot software on a host computer, enabling developers to simulate various UAV platforms. It can be controlled using GCS programs like Mission Planner or MAVProxy. Additionally, it supports user-defined programs written in C or Python that communicate via MAVLink, providing flexibility in customizing and automating UAV operations.

Gazebo [77]: This simulator was originally designed for 3D robotics systems but can also be used for simulating different UAVs. It can simulate complex 3D virtual environments, including sensor models such as cameras, lidars, or sonars, and supports multi vehicle simulation. Gazebo supports ROS integration and provides cloud connectivity.

FlightGear [78]: It is a flight simulator, and it supports multiple vehicles and aircraft in a realistic environment. It offers the possibility to simulate weather conditions, including thunderstorms, snow, and rain, and can simulate different types of atmospheric flows.

jMavSim [79]: It is a simulator developed by the Pixhawk team. It is easy to configure and simulate UAVs in various environments and conditions. However, it does not offer the possibility to integrate sensors other than those provided, nor can it be used for multi vehicle simulations. jMavSim supports ROS and can run SITL over UDP and HITL over a serial connection.

AirSim [80]: This simulator is developed by Microsoft and can simulate both ground and aerial vehicles in several environments. It can simulate weather conditions, with realistic views. It is aimed at developing and testing deep learning, computer vision, and reinforcement learning algorithms for autonomous vehicle applications. Interaction with the vehicle is performed through an API that can be programmed with C#, Java, and Python.

JSBSim [81]: This is an open-source flight dynamics library used to model flight dynamics of UAVs. It provides different physics models, such as aerodynamic models, propulsion, atmosphere models, etc., which work together to calculate the overall UAV dynamics. JSBSim can be run in a standalone batch mode flight simulator with no graphical displays for testing, or integrated with FlightGear, Unreal engine, and many other graphical simulation environments. The configuration is performed through XML files. Several popular simulators, such as FlightGear, use this simulator.

Webots [82]: One of the multi-platform applications is Cyberbotics' Webots, which enables the simulations of different robots, UAVs, unmanned underwater vehicles, etc. Webots can be used in the fields of education, research, and industry. The simulated agents can be programmed in different languages, including C, C++, Java or Python.

Table 6 provides the principal features of open-source simulators, and Figure 3 shows the Ardupilot-SITL controlled through the Mavproxy tool.

Several papers have utilized various tools to analyze specific aspects of UAV technology. For instance, the paper [83] presents the AI Wings system using Ardupilot combined with AirSim for controlling UAVs within an AIoT framework. Another paper [84] discusses the combination of Ardupilot and a Gazebo/ROS simulator for networked small UAVs. FlyNetSim, a system combining the ns-3 network simulator with Ardupilot SITL for synchronized UAV network simulations, is presented in the work [85]. The same combination is found in Ref. [86], where a UAV-based real-time wind-sensing mission planner module for maritime emission monitoring is tested. In [87], Gazebo is combined with Ardupilot for a HITL simulation environment to test UAVs, and similarly, in the paper [88], a Gazebo and PX4 HITL simulation of a time-delayed anti-swing controller for a quadrotor with a

suspended load is presented. A 3-DOF HITL Pixhawk-based test platform for controlling multi-rotor vehicles is designed in [89], emphasizing the importance of HITL simulations in UAV control system development. The paper [90] explores a 6DoF HITL test platform based on MATLAB-Simulink/FPGA for multi-rotor UAVs, demonstrating the comprehensive testing capabilities of HITL setups. Furthermore, reference [91] proposes a Gazebo-based HITL simulation to manage multi-rotor drones. The Paparazzi environment provides its Paparazzi Center [92], a comprehensive graphical interface that serves as the hub for the Paparazzi UAV project, allowing for JSBSim and FlightGear-based simulations. This hub provides SITL and HITL tools to simulate the flight environment on a computer, as evidenced in Refs. [93,94]. Several examples underline the wide use of Paparazzi simulations. For example, the paper [95] presents the use of the JSBSim tool to simulate maritime UAV-VTOL robot flight control, and in the work [96] FlightGear is used to simulate a fleet of drones. Furthermore, Ref. [97] describes a Paparazzi-based HITL simulation of a dragonfly flapping drone, Ref. [98] evaluates the robustness of panel method-based path planning under the effects of wind gusts, and Ref. [99] describes some augmented reality (AR) applications focused on safety. The rest of the autopilots do not have SITL and HITL simulations at the same level as the previous autopilots, but SITL simulations are possible to carry out. For example, LibrePilot has a plug-in for RealFlight, Betaflight has its own for Gazebo, and iNAV has two plug-ins for RealFlight and X-Plane. However, there are no relevant papers describing their use.

Table 6. Principal features of open-source simulators.

Simulator	Scripting Language	Platform	Autopilot	ROS
Ardu-SITL	C, Python	Win, Lin	Ardupilot	Yes
Gazebo	C++, Python	Win, Lin, Mac	Ardupilot, PX4, Paparazzi Betaflight,	Yes
FlighGear	Nasal, C++	Win, Lin, Mac	Ardupilot, PX4, Paparazzi	Yes
JMAVSIM	Mavlink	Win, Lin, Mac	PX4	Yes
AirSim	C#, Java, and Python	Win, Lin	Ardupilot, PX4	Yes
JSBSim	Xml	Win, Lin, Mac	Ardupilot, PX4, Paparazzi	Yes
Webots	C++, python, java	Win, Lin, Mac	Multiple	Yes

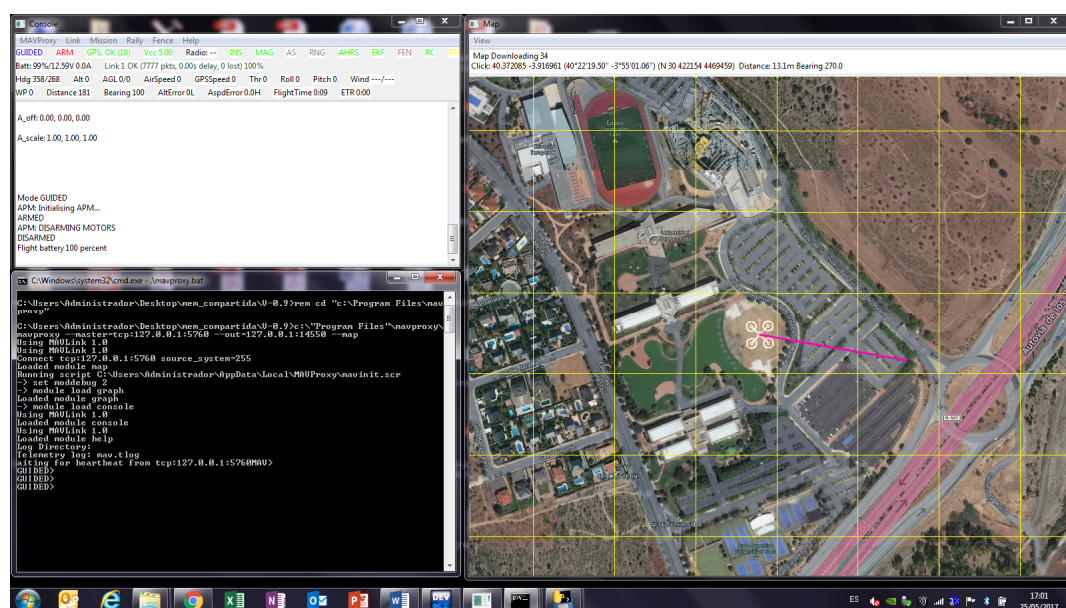


Figure 3. SITL controlled through MAVproxy tool.

5. Advantages and Disadvantages of Each Autopilot

Open-source autopilots offer advantages and disadvantages, catering to different needs and applications within the UAV community. Below is a discussion of the pros and cons of prominent autopilots, along with a summary table for quick reference.

Ardupilot is established for its versatility and extensive feature set, making it suitable for a wide range of UAV applications, from hobbyist projects to professional deployments. One of its significant advantages is its robust support for pairing with companion computers like the Raspberry Pi and NVIDIA Jetson, enabling advanced onboard processing and complex mission planning. The DroneKit API provides an exceptional advantage for Ardupilot developers. Ardupilot's compatibility with ROS middleware enhances its integration possibilities. The availability of both SITL and HITL tools provides a comprehensive environment for simulation and real-world testing, making it a preferred choice for academic institutions and researchers. However, its integration with MATLAB/Simulink PSP is not fully achieved compared to PX4, as it is limited to exchanging MAVLink messages and requires custom development due to the lack of native support. Another weakness is the firmware complexity and the steep learning curve, which can be challenging for beginners.

PX4 is a highly flexible and modular autopilot system, widely used in both academic and industrial settings. Its architecture supports a broad range of UAV types, from simple drones to complex autonomous systems. PX4 excels in pairing with companion computers, facilitating real-time data processing and advanced control strategies. The availability of MAVSDK API provides an exceptional advantage to PX4 developers for developing high level tasks. Its compatibility with ROS makes it an excellent platform for research and development, particularly in robotics and autonomous systems. The integration of PX4 with MATLAB/Simulink is seamless thanks to the MATLAB/Simulink PSP, a powerful toolbox for UAV applications development. This integration provides several benefits, such as rapid prototyping, advanced modeling, and real-time testing, which are particularly beneficial in academic settings where students and researchers can experiment with advanced control theories and autonomous systems in a safe and controlled manner. The extensive SITL and HITL support available for PX4 allows for advanced simulations and testing. However, like Ardupilot, PX4 can be complex to set up and configure, which may be a barrier for new users.

Paparazzi has a long-standing reputation, particularly in France. It is highly customizable and supports a variety of UAV platforms, making it suitable for experimental and novel UAV designs. Paparazzi's ability to pair with companion computers, though not as advanced as Ardupilot or PX4, provides significant flexibility for advanced applications. The Paparazzi Center is a versatile tool, offering a user-friendly interface for managing real flights as well as SITL and HITL simulation possibilities, making it a valuable resource for research. However, despite its compatibility with ROS middleware, there are no significant scientific papers testifying to the scope of this feature. The MATLAB/Simulink PSP toolbox is not compatible with Paparazzi, which constitutes a significant obstacle to extending its influence in new research and educational areas. Finally, its smaller user community may limit the availability of support and resources compared to more popular systems like Ardupilot and PX4.

LibrePilot, which is designed with a focus on reliability and ease of use, is an attractive option for hobbyists. It supports a range of UAV types, primarily multi-copters, and offers a straightforward setup process. While it can pair with companion computers, its capabilities in this area are not as advanced as those of Ardupilot or PX4. Its limited compatibility with ROS and MATLAB/Simulink PSP restricts its use in research applications. Although it provides SITL and HITL tools, these are not as comprehensive as those available for the other autopilots. Its smaller user community may limit the availability of support and resources.

Betaflight is specifically tailored for racing drones and acrobatic flying, known for its high-performance flight responsiveness. It supports pairing with companion computers, but its primary focus on high-speed performance means it may not be as versatile as other

applications. Its limited compatibility with MATLAB/Simulink PSP restricts its use in research and education. It does not offer extensive SITL or HITL tools, as its primary focus is on real-world performance rather than simulation. However, for FPV racing and acrobatic flying, Betaflight is unmatched in its performance, making it a popular choice in the racing community.

iNAV is designed for advanced navigation capabilities, supporting a variety of UAV types. It can be easily combined with companion computers, providing advanced navigation and control features. It has some compatibility with ROS, but it is not as extensive as Ardupilot or PX4, and its limited compatibility with MATLAB/Simulink PSP restricts its use in research and education. It also offers SITL and HITL tools, but they are not as comprehensive as those provided by Ardupilot or PX4. Its strength lies in its focus on navigation and control systems, making it a good choice for applications requiring precise navigation.

Table 7 summarizes the strengths and weaknesses of the open-source autopilots.

Table 7. Prominent autopilots strengths and weaknesses.

Autopilot	ROS	MATLAB PSP	SITL/HITL	Com. Computer	Educ/Research
Ardupilot	Strong	Moderate	Strong	Strong	Strong
PX4	Strong	Strong	Strong	Strong	Strong
Paparazzi	Moderate	Limited	Strong	Strong	Moderate
LibrePilot	Limited	Limited	Limited	Moderate	Limited
Betaflight	Limited	Limited	Limited	Limited	Limited
iNAV	Limited	Limited	Limited	Moderate	Limited

6. Conclusions

In this survey, prominent open-source autopilots have been compared by evaluating their impact on research and education, gathering evidence of their capabilities in streamlining UAV application development. The comparison focuses on their potential for pairing with companion computers, compatibility with ROS middleware and/or the MATLAB/Simulink environment, and the availability of simulation-in-the-loop (SITL) and hardware-in-the-loop (HITL) simulation tools. In summary, each open-source autopilot system has its own strengths and weaknesses, making them suitable for different applications and user expertise levels. ArduPilot and PX4 are highly versatile and well-supported, making them ideal for research and education, despite their complexity and steep learning curve. Paparazzi offers customization for experimental designs but lacks the advanced integration capabilities of ArduPilot and PX4, particularly with MATLAB/Simulink. LibrePilot, Betaflight, and iNAV cater to hobbyists, racing enthusiasts, and precise navigation, respectively, and offer a straightforward setup process. However, their limited compatibility with ROS and MATLAB/Simulink restricts their use in research and education. Selecting the right autopilot depends on the specific needs and goals of the UAV project.

Several promising directions for future work can enhance the capabilities and applications of open-source autopilots. First, addressing security concerns is critical; deploying reliable technologies to protect against cyber threats and unauthorized access will ensure safer UAV operations, particularly in sensitive or critical applications. Another area for exploration is the integration of advanced machine learning and AI techniques for autonomous decision-making and adaptive control, which could significantly extend UAV capabilities, especially in dynamic and uncertain environments. Additionally, promoting interoperability and standardization among different autopilot systems can enable more flexible and versatile UAV operations. Developing common communication protocols and interfaces will further enhance this interoperability. By pursuing these directions, the open-source autopilot community can continue to innovate and expand the potential applications of UAVs.

Funding: This research received no external funding.

Data Availability Statement: No new data were created or analyzed in this study. Data sharing is not applicable to this article.

Conflicts of Interest: The author declares no conflicts of interest.

References

1. ArduPilot. Available online: <https://ArduPilot.org/dev/index.html> (accessed on 10 November 2024).
2. Pixhawk. Available online: <https://pixhawk.org/> (accessed on 10 November 2024).
3. PX4. Available online: <https://px4.io/> (accessed on 10 November 2024).
4. Paparazzi. Available online: https://wiki.paparazziuav.org/wiki/Main_Page (accessed on 10 November 2024).
5. LibrePilot. Available online: <https://www.librepilot.org/site/index.html> (accessed on 10 November 2024).
6. Betaflight. Available online: <https://betaflight.com/> (accessed on 10 November 2024).
7. iNAV. Available online: <https://github.com/inavFlight/inav/wiki> (accessed on 10 November 2024).
8. Meier, L.; Honegger, D.; Pollefeys, M. PX4: A node-based multithreaded open-source robotics framework for deeply embedded platforms. In Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), Seattle, WA, USA, 26–30 May 2015; pp. 6235–6240. [CrossRef]
9. Drouin, A.; Brisset, P.; Jestin, Y. Reengineering the Paparazzi Autopilot Navigation System. *IFAC Proc.* **2007**, *40*, 270–275. [CrossRef]
10. Ardupilot-ChibiOS. Available online: <https://ardupilot.org/copter/docs/common-loading-chibios-firmware-onto-pixhawk.html> (accessed on 20 November 2024).
11. DoneKit-Python. Available online: <https://dronekit.netlify.app/about/overview> (accessed on 20 November 2024).
12. Lua-Scripting. Available online: <https://ardupilot.org/dev/docs/common-lua-scripts.html> (accessed on 20 November 2024).
13. PX4-NuttX. Available online: <https://docs.px4.io/main/en/concept/architecture.html#os-specific-information> (accessed on 20 November 2024).
14. MAVSDK. Available online: <https://mavsdk.mavlink.io/main/en/index.html> (accessed on 20 November 2024).
15. Paparazzi-ChibiOS. Available online: https://wiki.paparazziuav.org/wiki/RT_Paparazzi#Paparazzi_with_ChibiOS/RT (accessed on 20 November 2024).
16. PprzLink. Available online: <https://wiki.paparazziuav.org/wiki/Pprzlink> (accessed on 20 November 2024).
17. FreeRTOS. Available online: <https://librepilot.atlassian.net/wiki/spaces/LPDOC/pages/100523730/LibrePilot+System+Architecture> (accessed on 20 November 2024).
18. YAMSPy Protocol. Available online: <https://github.com/thecognifly/YAMSPy> (accessed on 20 November 2024).
19. MAVLink. Available online: <https://MAVLink.io/en/> (accessed on 10 November 2024).
20. Koubaa, A.; Allouch, A.; Alajlan, M.; Javed, Y.; Belghith, A.; Khalgui, M. Micro Air Vehicle Link (MAVlink) in a Nutshell: Survey. *IEEE Access* **2019**, *7*, 87658–87680. [CrossRef]
21. Allouch, A.; Cheikhrouhou, O.; Koubaa, A.; Khalgui, M.; Abbes, T. MAVSec: Securing the MAVLink Protocol for Ardupilot/PX4 Unmanned Aerial Systems. In Proceedings of the 2019 15th International Wireless Communications and Mobile Computing Conference, IWCMC 2019, Tangier, Morocco, 24–28 June 2019.
22. PprzLink. Available online: <https://pprzlink.readthedocs.io/en/latest/> (accessed on 20 November 2024).
23. UAVTalk. Available online: <https://librepilot.atlassian.net/wiki/spaces/LPDOC/pages/8552471/UAVTalk> (accessed on 20 November 2024).
24. MSP. Available online: <https://github.com/iNavFlight/inav/wiki/MSP-V2> (accessed on 20 November 2024).
25. APM Planner2. Available online: <https://ardupilot.org/planner2/> (accessed on 10 November 2024).
26. MissionPlanner. Available online: <https://ardupilot.org/planner/> (accessed on 10 November 2024).
27. Adropilot. Available online: <https://diydrones.com/group/andropilot-users-group> (accessed on 10 November 2024).
28. MAVProxy. Available online: <https://ardupilot.org/mavproxy/> (accessed on 10 November 2024).
29. QGroundControl. Available online: <http://qgroundcontrol.com/> (accessed on 10 November 2024).
30. Paparazzi Center. Available online: https://paparazzi-uav.readthedocs.io/en/latest/quickstart/paparazzi_center_tour.html (accessed on 20 November 2024).
31. MAVROS. Available online: <https://github.com/MAVLink/MAVROS> (accessed on 10 November 2024).
32. Perez, E.; Winger, A.; Tran, A.; Garcia-Paredes, C.; Run, N.; Ket, N.; Bhandari, S.; Raheja, A. Autonomous Collision Avoidance System for a Multicopter using Stereoscopic Vision. In Proceedings of the 2018 International Conference on Unmanned Aircraft Systems (ICUAS), Dallas, TX, USA, 12–15 June 2018; pp. 579–588.
33. De Cos, C.R.; Fernandez, M.J.; Sanchez-Cuevas, P.J.; Acosta, J.Á.; Ollero, A. High-level modular autopilot solution for fast prototyping of unmanned aerial systems. *IEEE Access* **2020**, *8*, 223827–223836. [CrossRef]
34. Wilson, A.N.; Kumar, A.; Jha, A.; Cenkeramaddi, L.R. Embedded sensors, communication technologies, computing platforms and machine learning for UAVs: A review. *IEEE Sens. J.* **2021**, *22*, 1807–1826. [CrossRef]

35. Kumar, R.H.; Vanjare, A.M.; Omkar, S.N. Autonomous Drone Navigation using Monocular Camera and Light Weight Embedded System. In Proceedings of the 2023 International Conference for Advancement in Technology (ICONAT), Goa, India, 24–26 January 2023; pp. 1–6.
36. Sinisterra, A.; Dhanak, M.; Kouvaras, N. A USV platform for surface autonomy. In Proceedings of the OCEANS 2017—Anchorage, Anchorage, AK, USA, 18–21 September 2017; pp. 1–8.
37. Singhania, P.; Siddharth, R.N.; Das, S.; Suresh, A.K. Autonomous navigation of a multirotor using visual odometry and dynamic obstacle avoidance. In Proceedings of the 2017 IARC Symposium on Indoor Flight Issues, Beijing, China, 18–20 August 2017.
38. Diez-Tomillo, J.; Alcaraz-Calero, J.M.; Wang, Q. Edge-accelerated UAV operations: A case study of open-source solutions. In Proceedings of the 2024 International Wireless Communications and Mobile Computing (IWCMC), Ayia Napa, Cyprus, 27–31 May 2024; pp. 114–119.
39. Pulungan, A.B.; Putra, Z.Y.; Sidiqi, A.R.; Hamdani, H.; Parigalan, K. Drone Kit-Python for Autonomous Quadcopter Navigation. *JOIV Int. J. Inform. Vis.* **2024**, *8*, 1287–1294. [\[CrossRef\]](#)
40. Koubâa, A.; Qureshi, B.; Sriti, M.-F.; Allouch, A.; Javed, Y.; Alajlan, M.; Cheikhrouhou, O.; Khalgui, M.; Tovar, E. Dronemap Planner: A service-oriented cloud-based management system for the Internet-of-Drones. *Ad Hoc Netw.* **2019**, *86*, 46–62. [\[CrossRef\]](#)
41. Kumar, G.P.; Sridevi, B. Chapter 6—Development of Efficient Swarm Intelligence Algorithm for Simulating Two-Dimensional Orthomosaic for Terrain Mapping Using Cooperative Unmanned Aerial Vehicles; Academic Press: Cambridge, MA, USA, 2020; pp. 75–93. [\[CrossRef\]](#)
42. Wu, C.H.; Tu, S.H.; Tu, S.W.; Wang, L.H.; Chen, W.H. Realization of Remote Monitoring and Navigation System for Multiple UAV Swarm Missions: Using 4G/WiFi-Mesh Communications and RTK GPS Positioning Technology. In Proceedings of the 2022 International Automatic Control Conference (CACs), Kaohsiung, Taiwan, 3–6 November 2022; pp. 1–6. [\[CrossRef\]](#)
43. Koulianos, A.; Litke, A. Blockchain technology for secure communication and formation control in smart drone swarms. *Future Internet* **2023**, *15*, 344. [\[CrossRef\]](#)
44. Soares, D.A.; Ramos, A.C.B.; da Costa Junior, R.A. Development of a Portable, Low-Cost System for Ground Control Station for Drones. In Proceedings of the Information Technology-New Generations: 14th International Conference on Information Technology, Las Vegas, NV, USA, 10–12 April 2018; Springer International Publishing: New York, NY, USA, 2018; pp. 767–771.
45. Villa, D.K.; Brandao, A.S.; Sarcinelli-Filho, M. A survey on load transportation using multirotor UAVs. *J. Intell. Robot. Syst.* **2020**, *98*, 267–296. [\[CrossRef\]](#)
46. Xu, Z.; Li, X.; Wang, X. Research on Ad Hoc Network Routing Protocol for UAV Application. In Proceedings of the International Conference on 5G for Future Wireless Networks, Harbin, China, 17–18 December 2022; Springer Nature Switzerland: Cham, Switzerland, 2022; pp. 75–83.
47. Patil, D.; Pournouri, S. Evaluating the Security of Open-Source Linux Operating Systems for Unmanned Aerial Vehicles. In Proceedings of the International Conference on Global Security, Safety, and Sustainability, London, UK, 11–12 October 2023; Springer Nature Switzerland: Cham, Switzerland, 2023; pp. 21–49.
48. Wang, H.; Chen, S.; Durak, U.; Hartmann, S. Simulation infrastructure for aeronautical informatics education. In Proceedings of the 50th Computer Simulation Conference, Bordeaux, France, 9–12 July 2018; pp. 1–12.
49. Chen, S.; Durak, U.; Hartmann, S. Modeling and Simulation-based Development of Autonomy Features for Drones. *Simul. Notes Eur.* **2018**, *28*, 55–60. [\[CrossRef\]](#)
50. Pise, A.; Bhandari, H.; Jadhav, K.; Dorge, S.; Mankar, G.; Yewale, P.M. Image processing-based drone for monitoring and surveillance. *Image* **2020**, *7*, 145–148.
51. Bigazzi, L.; Basso, M.; Boni, E.; Innocenti, G.; Pieraccini, M. A multilevel architecture for autonomous uavs. *Drones* **2021**, *5*, 55. [\[CrossRef\]](#)
52. Bigazzi, L.; Gherardini, S.; Innocenti, G.; Basso, M. Development of non-expensive technologies for precise maneuvering of completely autonomous unmanned aerial vehicles. *Sensors* **2021**, *21*, 391. [\[CrossRef\]](#) [\[PubMed\]](#)
53. Foehn, P.; Romero, A.; Scaramuzza, D. Time-optimal planning for quadrotor waypoint flight. *Sci. Robot.* **2021**, *6*, eabh1221. [\[CrossRef\]](#) [\[PubMed\]](#)
54. Córdoba, J.O.; Zarca, A.M.; Skármeta, A. Unmanned Aerial Vehicle Multi-Access Edge Computing as Security Enabler for Next-Gen 5G Security Frameworks. *Intell. Autom. Soft Comput.* **2023**, *37*, 2307–2333.
55. ROS. Available online: https://docs.px4.io/main/en/ros/ros2_comm.html (accessed on 10 November 2024).
56. XRCE-DDS. Available online: https://docs.px4.io/main/en/middleware/uxrce_dds.html (accessed on 10 November 2024).
57. Baldi, S.; Sun, D.; Xia, X.; Zhou, G.; Liu, D. ArduPilot-Based Adaptive Autopilot: Architecture and Software-in-the-Loop Experiments. *IEEE Trans. Aerosp. Electron. Syst.* **2022**, *58*, 4473–4485. [\[CrossRef\]](#)
58. Bernardeschi, C.; Fagiolini, A.; Palmieri, M.; Scrima, G.; Sofia, F. Ros/Gazebo based simulation of co-operative uavs. In Proceedings of the Modelling and Simulation for Autonomous Systems: 5th International Conference, MESAS 2018, Prague, Czech Republic, 17–19 October 2018; Revised Selected Papers 5. Springer International Publishing: New York, NY, USA, 2019; pp. 321–334.
59. Ma, C.; Zhou, Y.; Li, Z. A New Simulation Environment Based on Airsim, ROS, and PX4 for Quadcopter Aircrafts. In Proceedings of the 2020 6th International Conference on Control, Automation and Robotics (ICCAR), Singapore, 20–23 April 2020; pp. 486–490. [\[CrossRef\]](#)
60. Daspan, A.; Nimsongprasert, A.; Srichai, P.; Wiengchanda, P. Implementation of Robot Operating System in Raspberry Pi 4 for Autonomous Landing Quadrotor on ArUco Marker. *Int. J. Mech. Eng. Robot. Res.* **2023**, *12*, 210–215. [\[CrossRef\]](#)

61. Fernandez, M.J.; Sanchez-Cuevas, P.J.; Heredia, G.; Ollero, A. Securing UAV communications using ROS with custom ECIES-based method. In Proceedings of the 2019 Workshop on Research, Education and Development of Unmanned Aerial Systems (RED UAS), Cranfield, UK, 25–27 November 2019; pp. 237–246.
62. Schmittle, M.; Lukina, A.; Vacek, L.; Das, J.; Buskirk, C.P.; Rees, S.; Sztipanovits, J.; Grosu, R.; Kumar, V. OpenUAV: A UAV testbed for the CPS and robotics community. In Proceedings of the 2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS), Porto, Portugal, 11–13 April 2018; pp. 130–139.
63. Pprzros. Available online: <https://github.com/enacuavlab/pprzros> (accessed on 10 November 2024).
64. Sbus. Available online: https://github.com/LTU-RAI/mav_sbus_bridge (accessed on 10 November 2024).
65. Price, E.; Liu, Y.T.; Black, M.J.; Ahmad, A. Simulation and Control of Deformable Autonomous Airships in Turbulent Wind. In *Intelligent Autonomous Systems 16 (IAS 2021)*; Ang, M.H., Jr., Asama, H., Lin, W., Foong, S., Eds.; Lecture Notes in Networks and Systems; Springer: Cham, Switzerland, 2022; Volume 412. [CrossRef]
66. Wessendorp, N.; Dinaux, R.; Dupeyroux, J.; de Croon, G.C. Obstacle Avoidance onboard MAVs using a FMCW RADAR. In Proceedings of the 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Prague, Czech Republic, 21 September–1 October 2021; pp. 117–122.
67. MATLAB-PSP. Available online: <https://es.mathworks.com/help/supportpkg/px4/index.html> (accessed on 30 September 2024).
68. Wang, S.; Dai, X.; Ke, C.; Quan, Q. RflySim: A Rapid Multicopter Development Platform for Education and Research Based on Pixhawk and MATLAB. In Proceedings of the 2021 International Conference on Unmanned Aircraft Systems (ICUAS), Athens, Greece, 15–18 June 2021; pp. 1587–1594. [CrossRef]
69. Quan, Q.; Dai, X.; Wang, S. *Multicopter Design and Control Practice: A Series Experiments Based on MATLAB and Pixhawk*; Springer Nature: Berlin, Germany, 2020.
70. Hareha, A.; Bousbaine, A.; Josaph, A.K. A Hardware Implementation of 6DOF Quadcopter MATLAB/Simulink Controller Algorithm to an Autopilot. In Proceedings of the 10th International Conference on Power Electronics, Machines and Drives (PEMD 2020), Online, 15–17 December 2020; pp. 485–490. [CrossRef]
71. Jing, Y.; Wang, X.; Heredia-Juesas, J.; Fortner, C.; Giacomo, C.; Sipahi, R.; Martinez-Lorenzo, J. PX4 Simulation Results of a Quadcopter with a Disturbance-Observer-Based and PSO-Optimized Sliding Mode Surface Controller. *Drones* **2022**, *6*, 261. [CrossRef]
72. MATLAB-SITL. Available online: <https://ardupilot.org/dev/docs/sitl-with-MATLAB.html> (accessed on 10 November 2024).
73. Muller, Y.; Oshiro, S.; Motohara, T.; Kinjo, A.; Suzuki, T.; Wada, T. Underwater Acoustic Mavlink Communication for Swarming AUVs. *IJCSNS* **2021**, *21*, 277.
74. Ibrahim, N.A.; Zakaria, M.Y.; Kamal, A. Development of a Flight Test Data Analyzer for Pixhawk Autopilots. In Proceedings of the AIAA SCITECH 2023 Forum, National Harbor, MD, USA, 23–27 January 2023; Volume 2023, p. 0482.
75. Offermann, A.; De Miras, J.; Castillo, P. Software architecture for controlling in real time aerial prototypes. In Proceedings of the 2023 International Conference on Unmanned Aircraft Systems (ICUAS), Warsaw, Poland, 6–9 June 2023; pp. 493–498.
76. Ardu-SITL. Available online: <https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html> (accessed on 10 November 2024).
77. Gazebo. Available online: <https://gazebo.org/docs> (accessed on 10 November 2024).
78. Flightgear. Available online: <https://www.flightgear.org/about/> (accessed on 10 November 2024).
79. jMavSim. Available online: https://docs.px4.io/main/en/sim_jmavsim/ (accessed on 10 November 2024).
80. AirSim. Available online: <https://www.microsoft.com/en-us/research/project/aerial-informatics-robotics-platform/> (accessed on 10 November 2024).
81. JSBSim. Available online: <https://jsbsim.sourceforge.net/> (accessed on 10 November 2024).
82. Webots. Available online: <https://cyberbotics.com/doc/guide/introduction-to-webots> (accessed on 10 November 2024).
83. Lai, K.-T.; Chung, Y.-T.; Su, J.-J.; Lai, C.-H.; Huang, Y.-H. AI Wings: An AIoT Drone System for Commanding ArduPilot UAVs. *IEEE Syst. J.* **2023**, *17*, 2213–2224. [CrossRef]
84. Moon, S.; Bird, J.J.; Borenstein, S.; Frew, E.W. A Gazebo/ROS-based Communication-Realistic Simulator for Networked sUAS. In Proceedings of the 2020 International Conference on Unmanned Aircraft Systems (ICUAS), Athens, Greece, 1–4 September 2020; pp. 1819–1827. [CrossRef]
85. Baidya, S.; Shaikh, Z.; Levorato, M. Flynetsim: An open source synchronized uav network simulator based on ns-3 and ardupilot. In Proceedings of the 21st ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems, MSWIM'18, Montreal, QC, Canada, 28 October–1 November 2018; ACM: New York, NY, USA; pp. 37–45. [CrossRef]
86. Karachalios, T.; Moschos, P.; Orphanoudakis, T. Maritime Emission Monitoring: Development and Testing of a UAV-Based Real-Time Wind Sensing Mission Planner Module. *Sensors* **2024**, *24*, 950. [CrossRef]
87. Nguyen, K.D.; Ha, C. Development of Hardware-in-the-Loop Simulation Based on Gazebo and Pixhawk for Unmanned Aerial Vehicles. *Int. J. Aeronaut. Space Sci.* **2018**, *19*, 238–249. [CrossRef]
88. Omar, H.M. Hardware-In-the-Loop Simulation of Time-Delayed Anti-Swing Controller for Quadrotor with Suspended Load. *Appl. Sci.* **2022**, *12*, 1706. [CrossRef]
89. Hancer, M.; Bitirgen, R.; Bayezit, I. Designing 3-DOF Hardware-In-The-Loop Test Platform Controlling Multirotor Vehicles. *IFAC-PapersOnLine* **2018**, *51*, 119–124. [CrossRef]

90. Wang, H.; Azaizia, D.; Lu, C.; Zhang, B.; Zhao, X.; Liu, Y. Hardware in the loop based 6DoF test platform for multi-rotor UAV. In Proceedings of the 2017 4th International Conference on Systems and Informatics (ICSAI), Hangzhou, China, 11–13 November 2017; pp. 1693–1697.
91. Thebe, K.Z.; Jamisola, R.S.; Ramalepa, L.P. A novel approach to control four multi-rotor drones in cooperative paired control using relative Jacobian. *Robotica* **2023**, *41*, 3004–3021. [[CrossRef](#)]
92. Paparazzi Center. Available online: <https://wiki.paparazziuav.org/wiki/Simulation> (accessed on 10 November 2024).
93. Brisset, P.; Drouin, A.; Gorraz, M.; Huard, P.S.; Tyler, J. The paparazzi solution. In Proceedings of the MAV 2006, 2nd US-European Competition and Workshop on Micro Air Vehicles, Sandestin, FL, USA, 30 October–2 November 2006.
94. Coopmans, C.; Podhradsky, M.; Hoffer, N.V. Software-and hardware-in-the-loop verification of flight dynamics model and flight control simulation of a fixed-wing unmanned aerial vehicle. In Proceedings of the 2015 Workshop on Research, Education and Development of Unmanned Aerial Systems (RED-UAS), Cancún, Mexico, 23–25 November 2015; pp. 115–122.
95. Zhao, D.L.; Anvar, A.M. Modelling and Simulation of Maritime UAV-VTOL Robot Flight Control. *Appl. Mech. Mater.* **2012**, *152*, 1149–1154. [[CrossRef](#)]
96. Bailon-Ruiz, R.; Reymann, C.; Lacroix, S.; Hattenberger, G.; De Marina, H.G.; Lamraoui, F. System simulation of a fleet of drones to probe cumulus clouds. In Proceedings of the 2017 International Conference on Unmanned Aircraft Systems (ICUAS), Miami, FL, USA, 13–16 June 2017; pp. 375–382.
97. Lashgari, M.; Naghash, A. Hardware in the loop simulation and implementation of a dragonfly-like MAV using clap and fling mechanism. *Proc. Inst. Mech. Eng. Part C J. Mech. Eng. Sci.* **2023**, *237*, 3363–3377. [[CrossRef](#)]
98. Bilgin, Z.; Bronz, M.; Yavrucuk, I. Experimental evaluation of robustness of panel-method-based path planning for urban air mobility. In Proceedings of the AIAA AVIATION 2022 Forum, Chicago, IL, USA, 27 June–1 July 2022; p. 3509.
99. Garcia, J.; Brock, A.; Saporito, N.; Hattenberger, G.; Paris, X.; Gorraz, M.; Jestin, Y. Designing human-drone interactions with the Paparazzi UAV System. In Proceedings of the 1st International Workshop on HumanDrone Interaction (CHI'19), Glasgow, UK, 4–9 May 2019.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.